

Beágyazott információs rendszerek

dr. Kovácsházy Tamás
7. labor forgatókönyv



Mesterséges Intelligencia és
Rendszertervezés Tanszék

GPIO használata

Bevezetéd, bemenet kezelése

AP PIN-ek

Ismétlés előadásról!

- Lásd adatlap és programozási leírás
- PIN-nek maximum 8 funkciója van (0-7)
 - Egy funkció: 8 A/D PIN-t, memória, stb.
 - Soknak kevesebb mint 8, 3-4 tipikusan
 - A legtöbbnek 8 funkciója van
 - Ezek a funkciók lehetnek a PRU funkciók is
 - Vagy a Linux, vagy egy PRU kezeli a kérdéses PIN-t
- Megadott funkció RESET után, tipikusan a 7-es (GPIO)
- Többnyire alapértelmezetten a PIN-ek magas impedanciájú (input) módban vannak RESET után
 - Egy pullup vagy pulldown ellenállással
- Ajánlott a CAPE-eken a tápot programozottan kapcsolni
 - Egy vagy több GPIO PIN-nel vagy egy PMIC-cel a CAPE-en
- SW konfiguráció: devicetree (boot + overlay) és/vagy config-pin program (futási idő)

AP PIN-ek

- Lásd adatlap és programozási leírás
- PIN-nek maximum 8 funkciója van (0-7)
 - Egy funkció: 8 A/D PIN memória, stb.
 - Soknak kevesebb mint 8 funkciója van tipikusan
 - A legtöbbnek 8 funkciója van
 - Ezek a funkciók lehetnek konfigurálhatóak is

Ismétlés előadásról!

- Vagy a Linux

- Megadott funkció

- Többnyire alap (input) módban

- Egy pullup vagy pull-down ellenállással

- Ajánlott a CAPE-eken a tápot programozottan kapcsolni

- Egy vagy több GPIO PIN-nel vagy egy PMIC-cel a CAPE-en

- SW konfiguráció: devicetree (boot + overlay) és/vagy config-pin program (futási idő)

Adatlap: 4.3 Signal descriptions (50. oldal)
Ref. Man.: 9.2.2 Pad Control Registers (1449. oldal)

CAPE csatlakozó alapértelmezett funkciók

Ismétlés előadásról!

P9

DGND	1	2	DGND
VDD_3V3	3	4	VDD_3V3
VDD_5V	5	6	VDD_5V
SYS_5V	7	8	SYS_5V
PWR_BUT	9	10	SYS_RESETN
UART4_RXD	11	12	GPIO_60
UART4_TXD	13	14	EHRPWM1A
GPIO_48	15	16	EHRPWM1B
SPI0_CS0	17	18	SPI0_D1
I2C2_SCL	19	20	I2C2_SDA
SPI0_D0	21	22	SPI0_SCLK
GPIO_49	23	24	UART1_TXD
GPIO_117	25	26	UART1_RXD
GPIO_115	27	28	SPI1_CS0
SPI1_D0	29	30	GPIO_112
SPI1_SCLK	31	32	VDD_ADC
AIN4	33	34	GNDA_ADC
AIN6	35	36	AIN5
AIN2	37	38	AIN3
AIN0	39	40	AIN1
GPIO_20	41	42	ECAPPWM0
DGND	43	44	DGND
DGND	45	46	DGND



LEGEND

POWER/GROUND/RESET

AVAILABLE DIGITAL

AVAILABLE PWM

SHARED I2C BUS

RECONFIGURABLE DIGITAL

ANALOG INPUTS (1.8V)

P8

DGND	1	2	DGND
MMC1_DAT6	3	4	MMC1_DAT7
MMC1_DAT2	5	6	MMC1_DAT3
GPIO_66	7	8	GPIO_67
GPIO_69	9	10	GPIO_68
GPIO_45	11	12	GPIO_44
EHRPWM2B	13	14	GPIO_26
GPIO_47	15	16	GPIO_46
GPIO_27	17	18	GPIO_65
EHRPWM2A	19	20	MMC1_CMD
MMC1_CLK	21	22	MMC1_DAT5
MMC1_DAT4	23	24	MMC1_DAT1
MMC1_DAT0	25	26	GPIO_61
LCD_VSYNC	27	28	LCD_PCLK
LCD_HSYNC	29	30	LCD_AC_BIAS
LCD_DATA14	31	32	LCD_DATA15
LCD_DATA13	33	34	LCD_DATA11
LCD_DATA12	35	36	LCD_DATA10
LCD_DATA8	37	38	LCD_DATA9
LCD_DATA6	39	40	LCD_DATA7
LCD_DATA4	41	42	LCD_DATA5
LCD_DATA2	43	44	LCD_DATA3
LCD_DATA0	45	46	LCD_DATA1

GPIO PIN-ek használata

- Előző laboron kimenetként használtuk
- Sajnos nem látjuk az elektromágneses teret ezen az hullámhosszon (frekvencián) sem a feszültséget
 - A látható fény kicsit más hullámhosszú
 - Nem is látnánk kb. 20-30 Hz feletti változásokat
 - A feszültséget sem látjuk...
- Kellenek műszerek
 - Legegyszerűbb: LED + ellenállás + vezeték, lassú 20-30 Hz
 - Multiméter, lassú, 20-30 Hz
 - Logikai analizátor vagy oszcilloszkóp (20+ darab kéne)
 - Később majd megnézzük így is...
- Nyomógomb (bemenet)
 - Most jön...

PIN konfiguráció lekérdezése

- A gpioinfo kipróbálása (generikus, chardev)
- A show-pins (TI specifikus)
 - A „show-pins | less” nem működik jól (színes karakter escape szekvenciák kezelésén elbukik)
 - „show-pins | less -R” működik (R = repaint the screen, feldolgozza az escape szekvenciákat)
 - A „show-pins | more” szintén jól működik
- Nézzük meg a kimeneteket...

A gpioinfo kimenté mit jelent?

- Kiírás: gpiochip-enként (4 32 bites van a BBB-ben)
 - Az első (gpiochip0 mindig kap tápot, a továbbiak (1-3) csak „power on” módban
 - Ha alszik a rendszer, amire fel kell ébrednie, azt a gpiochip0-ra kell kötni
 - Első sor CAPE azonosító + alapfunkció (a korai BBB-ék konfigurációja szerint), CAPE azonosító (ha nem érhető el, akkor unused), aktuális státusz (input/output), és felhúzó/lehúzó ellenállás konfigurációja
 - CAPE azonosító: P9_NUM/P8_NUM vagy P9.NUM/P8.NUM
 - Ezt a konvenciót más SW-ék és a doksik is használják

gpiochip0 - 32 lines:

```
line 0: "[mdio_data]" unused input active-high
line 1: "[mdio_clk]"      unused input active-high
line 2: "P9_22 [spi0_sclk]" "P9_22" input active-high [used]
line 3: "P9_21 [spi0_d0]" "P9_21" input active-high [used]
line 4: "P9_18 [spi0_d1]" "P9_18" input active-high [used]
line 5: "P9_17 [spi0_cs0]" "P9_17" input active-high [used]
```


A show-pins kimenete mit jelent?

- Beaglebone specifikus
- P8.22 / eMMC d5 5 fast rx up 1 mmc 1 d5
mmc@481d8000 (pinmux_emmc_pins)
- P8.03 / eMMC d6 6 fast rx up 1 mmc 1 d6
mmc@481d8000 (pinmux_emmc_pins)
- P8.04 / eMMC d7 7 fast rx up 1 mmc 1 d7
mmc@481d8000 (pinmux_emmc_pins)
- P8.19 8 fast rx down 7 gpio 0.22 << lo P8_19
(pinmux_P8_19_default_pin)
- P8.13 9 fast rx down 7 gpio 0.23 << lo P8_13
(pinmux_P8_13_default_pin)
- P8.14 10 fast rx down 7 gpio 0.26 << lo P8_14
(pinmux_P8_14_default_pin)
- P8.17 11 fast rx down 7 gpio 0.27 << lo P8_17 (pinm:

GPIO PIN-ek használata

- Használjuk bemenetként
 - Várni kell, hogy esemény (pl. gomb lenyomása) érkezzon
- Nyomógomb (bemenet)
 - GPIO66 és GPIO77
 - Lásd: EDUCAPE kapcsolási rajz (az ilyesminek az értelmezését is tanuljuk)
 - https://home.mit.bme.hu/~khazy/bir2026/educape_v01_schematic.pdf

Várakozás időintervallumra

- A `sleep()`, `usleep()` és `nanosleep` hívások
- Visszatekintés:
 - Mi történik ezekkel jelzések kezelése közben?
 - Mit okoznak a jelzések a blokkoló rendszerhívásoknál?
- Hogyan tudjuk kiírni az aktuális időt?
 - A `time()` fv és a `time_t` típus (seconds since Thu Jan 1 00:00:00 1970)
 - UNIX timestamp
 - Timespec struktúra
 - `tv_sec` : másodpercek az epoch óta
 - `tv_nsec` : ns a másodperc vége óta (felbontás fizikailag az órajel által korlátozott)
 - A `clock_gettime()` fv és a `CLOCK_MONOTONIC_RAW`, `CLOCK_MONOTONIC`, `CLOCK_REALTIME`

GPIO használata bemenetként

- Használjuk a sysfs-alapú GPIO elérést
- Első feladat:
 - A GPIO67-re (P8 felső 2x4 pin) csatlakozó nyomógomb állapotának a lekérdezése
 - Folyamatosan lekérdezés (polling)
 - Beállítás bemenetként (melyik az alapértelmezett?)
 - Működik?

Miért nem működik?

- Kiolvastuk a file-ban lévő karaktert, a file pointer a file-nak a végére mutat
 - Vissza kell állítani a file elejére...
 - Erre az lseek rendszerhívás szolgál
 - Nézzük meg a man page-t (man 2 lseek)
 - lseek(fd67,0, SEEK_SET)

CPU felhasználás?

- Várakozás nélküli megvalósítást csináltunk
 - Mekkora CPU felhasználás (másik SSH és benne pl. top vagy htop)
 - Mi a lekérdezési frekvencia?
 - Módosítsuk a meghatározására a programot!
- Használjuk a várakozó függvényeket pl. 1ms-os lekérdezés gyakoriságra
 - Mekkora CPU felhasználás (másik SSH és benne pl. top)

GPIO kezelése IT-vel + poll()-al

- A BBB GPIO tud IT-t kérni
 - Reference manual-ban megtalálhatók a HW részletek
 - SW részletek:
<https://www.kernel.org/doc/Documentation/gpio/sysfs.txt>
 - A sysfs az IT-es kezelési megoldást támogatja...
 - File-ra lehet blokkolni poll()-al
- Parancs: `echo 'both' > /sys/class/gpio/gpio67/edge`
 - "none", "rising", "falling", "both,,
- Utána poll()
 - Megírjuk a GPIO67-re együtt
 - Mindenki kibővíti 2 GPIO-ra (GPIO66)

A poll() rendszerhívás

Ismétlés előadásról!

- `int poll(struct pollfd *fds, nfds_t nfds, int timeout);`
 - Megadjuk neki azokat a file leírókat (descriptorokat), amire várakozni akarunk és azt, hogyan akarunk rájuk várakozni (pollfd struktúra)
 - Megadjuk a file leírók számát (nfds)
 - Meg egy timeout-ot is megadhatunk
- Visszatér, és ki tudjuk találni, hogy mi történt, melyik file leíró kezelhető/kezelendő:
 - A visszatérési érték
 - -1: hiba
 - 0: timeout
 - >0: A visszatérési érték a kezelhető file descriptorok száma
 - A pollfd struktúrában megadott eseményeket kezeljük egy alkalmas vezérlési szerkezetben
- Lesz a laborban...

Miért éppen a poll()?

Ismétlés előadásról!

- A select() programozási felülete archaikus és nehezen használható
 - Része a POSIX szabványnak, ezért más operációs rendszerekben is elérhető, a program könnyen portolható
 - Új szoftverben nem ajánlott a használata, de régi kód karbantartása során találkozhatunk vele
- A poll() egyszerűbben használható és hatékonyabb a select()-nél
 - Része a POSIX szabványnak, és ezért portábilis, sok helyen használják
- Az epoll() sokkal összetettebb, de egyben még hatékonyabb
 - Viszont csak a Linux-ban található meg
 - Más OS-ekben vannak hasonló API-k (BSD kqueue)

A poll() használata esetén

Ismétlés előadásról!

- Eseménykezelés, esemény-vezérelt szoftver architektúra
- Szekvenciális program
 - Az egyes file leírókhoz megadhatók az azokat kezelő függvények írhatóság és olvashatóság esetén
 - Azok teljesen lefutnak (aktivitás), mielőtt a poll()-t újra hívhatnánk
 - Így kell a szoftvert megírni
 - Lényegében a file leírókat érintő eseményekre tudunk esemény kezelőket írni
 - Plusz timeout
 - Ezzel az idő múlása is kezelhető a szoftverben
- Lényegében egy véges automatát írhatunk a poll()-al, ami eseményekre reagál
 - Ideális egyprocesszoros kiteljesítményű beágyazott Linux rendszerekben
 - Lassú, egymagos CPU, kevés memória, stb.